

halfedge：强类型 ID 模块设计文档

src/ids.rs

halfedge 库

2026-07-04

目录

1 设计目标	1
2 方案选型	2
2.1 候选方案对比	2
2.2 核心代码	2
3 版本号与失效判定	2
4 类型不可互换性	3
5 EdgeId：无向边的规范代表	3
5.1 定义	3
5.2 规范代表元	3
5.3 与 slotmap Key 的区别	4
6 单元测试	4
7 与 storage.rs 的衔接	4

1 设计目标

半边网格（Half-Edge Mesh）中存在三类截然不同的拓扑元素：顶点、半边、面。若所有元素都用同一个整数索引表示，把「顶点 ID」误传给「半边 ID」形参的 bug 会在运行期才暴露，调试代价高昂。本模块的目的是在**编译期**就把三种句柄彻底分开，使类型系统替我们挡住这类误用。

具体要求：

- 定义 VertexId、HalfEdgeId、FaceId；
- 自动派生 Debug、Copy、Eq、Ord；
- 与 slotmap 无缝衔接，可直接作为 SlotMap 的键。
- 提供 EdgeId 表示无向边（twin 对的规范代表元），用于无向边迭代、属性键等场景。

2 方案选型

2.1 候选方案对比

方案	优点	缺点
普通整数 u32+ 类型别名	写法最简单	别名仅是文档，编译期不可区分
Newtype struct VertexId(u32)	类型独立	需手写 trait、未带版本号
slotmap::new_key_type!	编译期强类型 + 版本号 + 自带全套 trait	引入 slotmap 依赖（已选用）

由于存储层（storage.rs）本就要使用 slotmap，第三种方案与项目其余部分天然吻合，且版本号机制是后续「删除自动失效」能力的基础，因此本模块直接采用 new_key_type! 宏。

2.2 核心代码

```
1 use slotmap::new_key_type;
2
3 new_key_type! {
4     pub struct VertexId;
5     pub struct HalfEdgeId;
6     pub struct FaceId;
7 }
```

new_key_type! 宏展开后等价于为每个类型派生：

$\underbrace{\text{Debug, Copy, Clone}}_{\text{需求 1 显式要求}} \cup \underbrace{\text{Eq, PartialEq, Ord, PartialOrd, Hash, Default}}_{\text{额外免费}}$

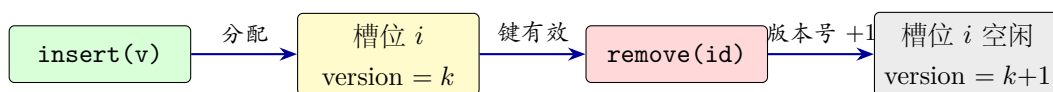
每个 key 的内部表示为

$\text{Key} = (\underbrace{\text{idx} : u32}_{\text{槽位下标}}, \underbrace{\text{version} : u32}_{\text{版本号}})$

共 64 位。

3 版本号与失效判定

slotmap 内部为每个槽位维护一个单调递增的版本号，元素的插/删/查都通过 (idx, version) 二元组完成：



形式化地，给定句柄 $h = (i, k)$ 与当前槽位版本 v_i ，有效性判定为

$$\text{valid}(h) \iff k = v_i \wedge \text{slot}_i \text{ occupied.}$$

这一性质是 `storage.rs` 中「删除即自动失效」的语义基础，本模块不需要再额外维护任何 tombstone 位。

4 类型不可互换性

三种 ID 在编译期就是不同类型，下列赋值会触发类型错误而非运行期失败：

```
1 let _: VertexId = HalfEdgeId::default(); //
2 let _: HalfEdgeId = FaceId::default(); //
```

这一约束完全由 Rust 的 nominal type 系统在零运行期开销下保证。

5 EdgeId：无向边的规范代表

`HalfEdgeId` 是有向的：每条无向边对应两条互为 twin 的半边。但在迭代边、按边存储属性等场景中，需要一个无向边的唯一标识。`EdgeId` 即为此设计。

5.1 定义

```
1 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
2 pub struct EdgeId(pub(crate) HalfEdgeId);
3
4 impl EdgeId {
5     pub fn halfedge(self) -> HalfEdgeId { self.0 }
6
7     pub fn vertices(&self, mesh: &MeshStorage)
8         -> Option<(VertexId, VertexId)>
9     {
10         let he = mesh.get_halfedge(self.0)?;
11         let dst = he.vertex;
12         let src = mesh.get_halfedge(he.twin)?.vertex;
13         Some((src, dst))
14     }
15 }
```

5.2 规范代表元

twin 对 $\{h, h^*\}$ 中，取 slotmap key 较小者作为「规范半边」，`EdgeId` 包装此规范半边。对于边界边 ($h^* = \text{None}$)，直接使用 h 自身作为代表。这保证：

- 同一条无向边对应唯一的 `EdgeId`；
- `EdgeId` 可作为 `HashMap` / `HashSet` 的键 (`Hash + Eq`)；
- 通过 `halfedge()` 取出 `HalfEdgeId` 后即可参与存储层 CRUD。

5.3 与 slotmap Key 的区别

VertexId、HalfEdgeId、FaceId 都是 slotmap 的 Key，可直接用于 SlotMap::get / insert / remove。EdgeId 不是 slotmap 的 Key：它不索引任何槽位，只是 HalfEdgeId 的语义包装。对 EdgeId 调用 mesh.get_halfedge(edge.halfedge()) 才能取回半边数据。

6 单元测试

```
1 #[test]
2 fn ids_are_copy_and_defaultable() {
3     let v: VertexId = VertexId::default();
4     let h: HalfEdgeId = HalfEdgeId::default();
5     let f: FaceId = FaceId::default();
6
7     let v2 = v; // Copy clone
8     assert_eq!(v, v2);
9     assert!(v <= v2); // Ord
10 }
11
12 #[test]
13 fn ids_have_distinct_types() {
14     //
15     // let _: VertexId = HalfEdgeId::default();
16     let _v: VertexId = VertexId::default();
17     let _h: HalfEdgeId = HalfEdgeId::default();
18     let _f: FaceId = FaceId::default();
19 }
```

两个测试分别覆盖了「trait 行为正确」与「类型不可互换」两点。

7 与 storage.rs 的衔接

本模块产出的三种 Id 类型直接作为 SlotMap 的键：

SlotMap<VertexId, Vertex>, SlotMap<HalfEdgeId, HalfEdge>, SlotMap<FaceId, Face>.

也就是说，VertexId 同时承担「用户面 ID」与「slotmap Key」双重身份，不需要再额外包一层 newtype，存储层的 VertexKey 概念即由 VertexId 实现。